

MIT School of Computing

MITADT University, Pune

Intelligent Classroom Energy Monitoring System

AI + IoT Capstone Project — Data Pipeline, ML Models & Backend

Project Code: BCAIAA22

Faculty Guide: Dr. Tushar Mote

Team: Aditya Bade (Data + AI + Backend) | Aditya Nalge | Aryan Dhondage | Om Teke

September 2026

1. Project Overview

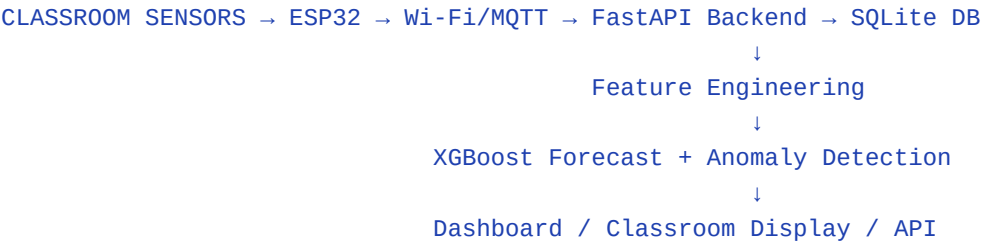
This document covers the complete data pipeline, AI/ML models, anomaly detection system, database design, and FastAPI backend built for the Intelligent Classroom Energy Monitoring project. It serves as the technical handover document for all team members.

1.1 Responsibility Split

Component	Owner	Status
CT clamp / PZEM sensors, wiring	Hardware Team (Nalge, Dhondage, Teke)	In Progress
ESP32 firmware + MQTT/HTTP publish	Hardware Team	In Progress
Data pipeline + feature engineering	Aditya Bade	✓ Complete
ML models (LR, RF, XGBoost, LSTM)	Aditya Bade	✓ Complete
Anomaly detection system	Aditya Bade	✓ Complete
Database schema (SQLite/PostgreSQL)	Aditya Bade	✓ Complete
FastAPI backend (5 endpoints)	Aditya Bade	✓ Complete
Live dashboard (HTML/JS)	Aditya Bade	✓ Complete
ESP32 → API integration	All members	Pending hardware

1.2 Full System Architecture

The complete data flow from physical classroom to dashboard:



2. Dataset — UCI Appliances Energy Prediction

 Important: This is a household dataset used as a proxy. It is NOT classroom data. All results in this document come from this public dataset. Models will be retrained on real ESP32 data when available.

2.1 Dataset Summary

Property	Value
Source	UCI Machine Learning Repository (ID: 374)
Location	Belgian house, Stambruges
Period	January 11 – May 27, 2016 (137 days)
Sampling interval	Every 10 minutes
Total rows	19,735
Total columns	29 (27 after dropping noise)
Missing values	0 — completely clean
Duplicate rows	0
Time gaps	0 — perfectly continuous

2.2 Column Dictionary

Column	Unit	Meaning	Use in Project
Appliances	Wh	Total appliance energy per 10-min interval	 Target variable (y)
lights	Wh	Lighting energy per 10-min interval	Feature → maps to light_power_w
T1–T9	°C	Temperature in 9 rooms	Feature → maps to temperature sensor
RH_1–RH_9	%	Relative humidity in 9 rooms	Feature → maps to humidity sensor
T_out	°C	Outdoor temperature	Feature → contextual
RH_out	%	Outdoor humidity	Feature → contextual
Windspeed	m/s	Wind speed from weather station	Feature
Visibility	km	Visibility distance	Feature
Tdewpoint	°C	Dew point temperature	Feature
Press_mm_hg	mmHg	Atmospheric pressure	Feature
rv1, rv2	—	Random noise variables	 Dropped — pure noise

Column	Unit	Meaning	Use in Project
		(intentional)	

2.3 Target Variable Analysis — Appliances

Statistic	Value
Minimum	10 Wh
25th percentile (Q1)	50 Wh
Median	60 Wh
Mean	97.7 Wh
75th percentile (Q3)	100 Wh
Maximum	1,080 Wh
Standard deviation	102.5 Wh
Skewness	3.39 (right-skewed)
Unique values	92 (multiples of 10 — sensor resolution)

2.4 Classroom Feature Mapping

Public Dataset Feature	Future Classroom ESP32 Feature
Appliances (Wh)	total_power_w × 10min → energy_kwh
lights (Wh)	light_power_w
T1, T2 ... T9 (°C)	temperature (DHT22 sensor)
T_out (°C)	outdoor temperature (optional)
RH_1 ... RH_9 (%)	humidity (DHT22 sensor)
hour, dayofweek	derived from ESP32 timestamp
is_weekend	derived from timestamp + class schedule
lag_1, lag_2, lag_3 ...	previous readings fetched from database
roll_mean_6, roll_mean_12	computed from database history
[NOT IN DATASET]	fan_power_w — new feature from ESP32
[NOT IN DATASET]	projector_power_w — new feature from ESP32
[NOT IN DATASET]	occupancy — PIR sensor / manual entry
[NOT IN DATASET]	light_intensity — BH1750 lux sensor
[NOT IN DATASET]	class_schedule (0/1 flag from timetable)

3. Exploratory Data Analysis (EDA)

3.1 Energy Consumption Over Time

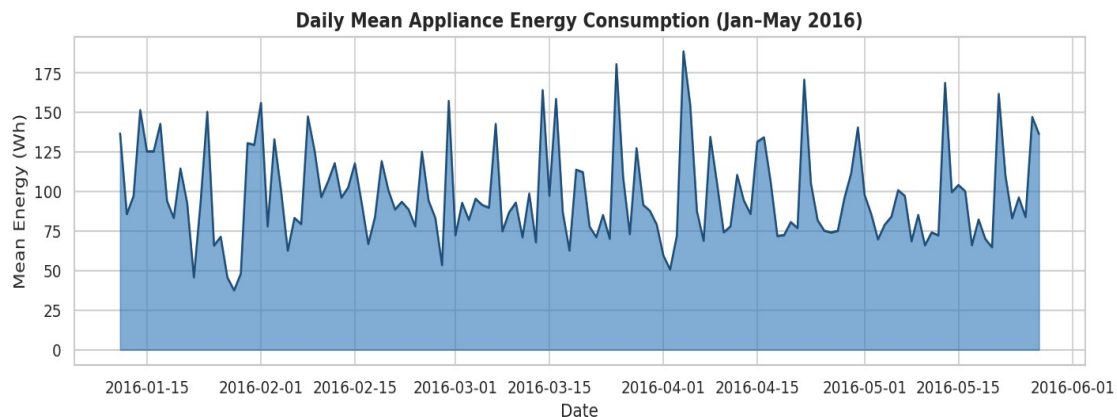


Figure 1 — Daily mean appliance energy consumption (Jan-May 2016)

The daily mean ranges from 55–188 Wh per 10-minute interval. Clear multi-day high-activity clusters are visible, confirming that time-based features and lag values will be valuable predictors. January (mean 97.0 Wh) and May (mean 94.2 Wh) are similar — this household in Belgium does not show strong seasonal variation over this 4.5-month period.

3.2 Hour-of-Day Consumption Pattern

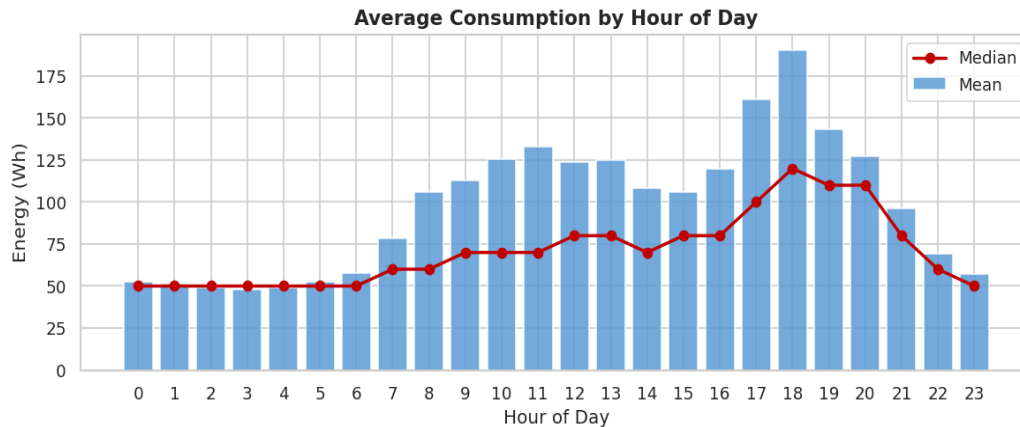


Figure 2 — Average energy consumption by hour of day

This is the most important pattern in the dataset. Night hours (00:00–06:00) show a low baseline of ~49–53 Wh (standby loads). A sharp morning ramp-up begins at 07:00 and reaches an evening peak at 18:00 of 190.4 Wh — likely due to cooking, television, and lighting all running simultaneously. The hour-of-day feature is the single most powerful predictor in our models.

3.3 Day-of-Week Pattern

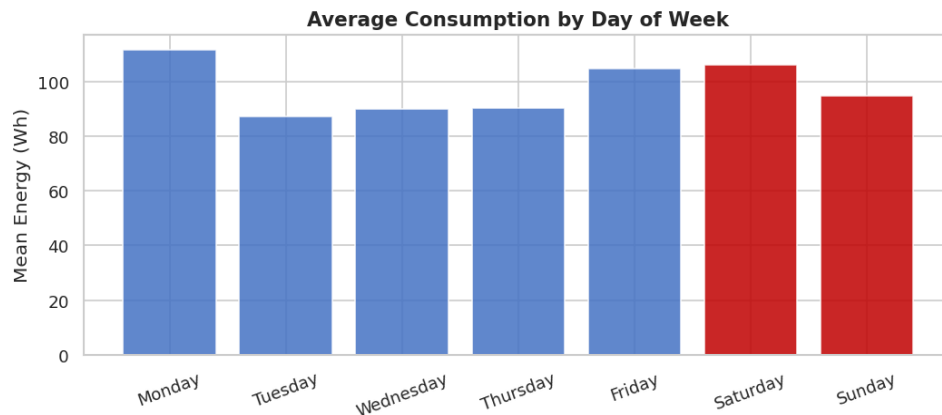


Figure 3 — Average consumption by day of week (blue = weekday, red = weekend)

Monday (111.5 Wh) and Friday (104.6 Wh) are highest. Tuesday–Thursday are lower (~87–90 Wh). The weekend effect is weaker than expected for this household. For a classroom system, this pattern will be very different — weekends should show near-zero consumption (no classes).

3.4 Correlation Analysis

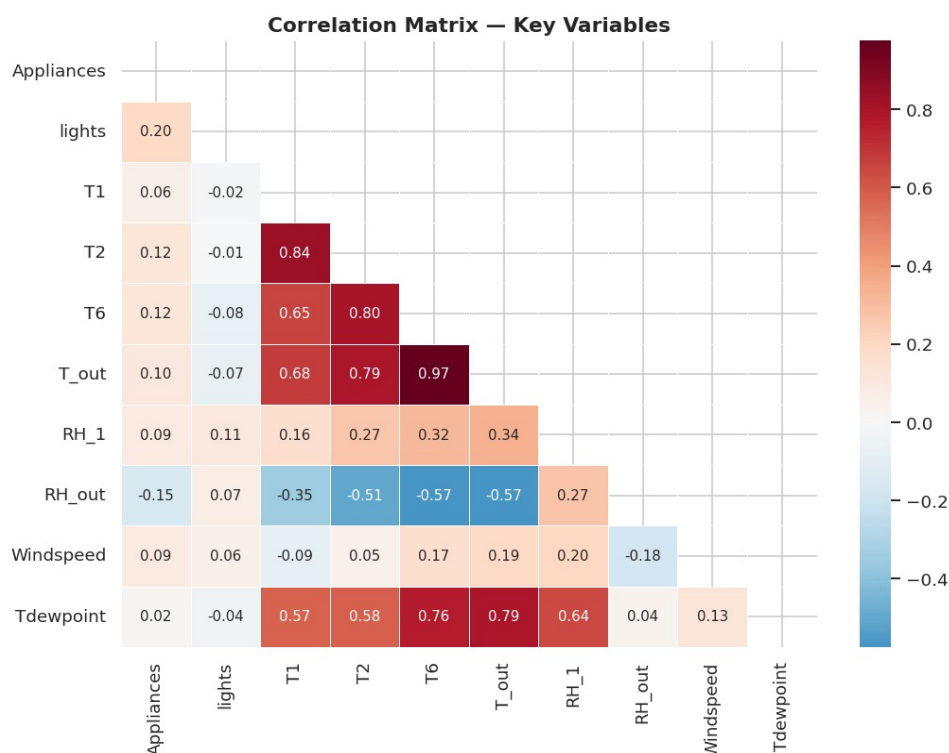


Figure 4 — Correlation matrix of key variables

No single feature has strong linear correlation with Appliances. The highest is lights ($r = +0.197$), followed by RH_out ($r = -0.152$). This confirms that linear models alone will not perform well — the consumption patterns are non-linear and time-dependent. This is exactly why tree-based models and LSTM are needed.

3.5 Target Variable Distribution

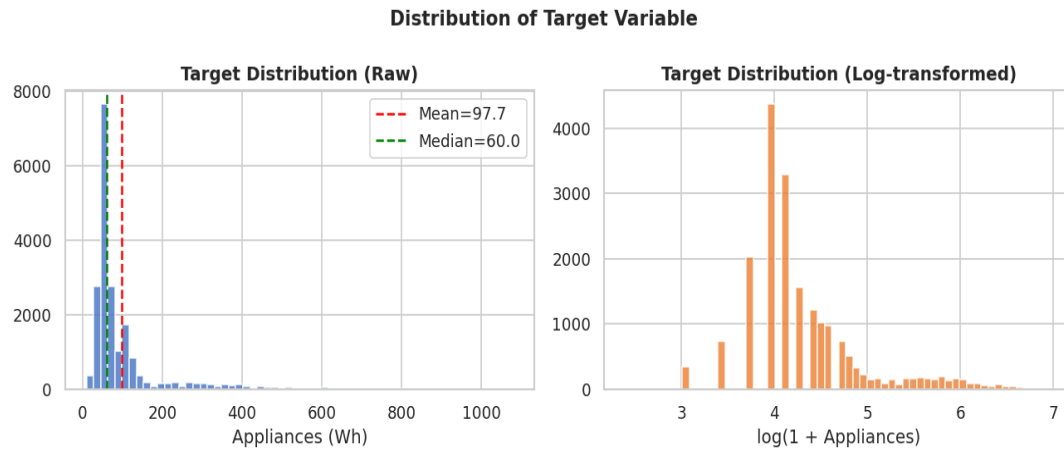


Figure 5 — Raw (left) and log-transformed (right) distribution of Appliances

The raw distribution has a skewness of 3.39 — heavily right-skewed. Most intervals cluster at 50–100 Wh, but rare peaks extend to 1,080 Wh. After log-transformation, skewness drops to 1.19. All outliers were retained as real energy events — they represent legitimate high-consumption periods (e.g., all appliances running simultaneously on a winter evening).

4. AI Problem Definition & Methodology

4.1 Problem Statement

Given the last N time steps of energy readings and contextual features, predict the appliance energy consumption at the NEXT 10-minute interval.

Element	Definition
Target (y)	Appliances energy at t+1 (next 10-minute interval, in Wh)
Features (X)	Lagged energy values, time features, lights, temperature, humidity
Prediction horizon	1 step ahead = 10 minutes into the future
Why 10-min ahead?	Practical for real-time dashboard; can chain for hourly estimates

4.2 Feature Engineering (29 Features Total)

Feature Group	Features Created	Reason
Cyclical time	hour_sin, hour_cos, dow_sin, dow_cos	Encodes circular nature of time (23→0, Sun→Mon)
Raw time	hour, dayofweek, is_weekend, month	Direct calendar signals for tree models
Lag energy	lag_1, lag_2, lag_3, lag_6, lag_12, lag_144, lag_1008	10min, 20min, 30min, 1hr, 2hr, 1day, 1week ago
Rolling stats	roll_mean_6, roll_mean_12, roll_std_6, roll_mean_144	1hr/2hr/24hr rolling average and std
Lights	lights, lights_lag1	Correlated with appliance use
Environmental	T1, T2, T6, T_out, RH_1, RH_out, Windspeed, Tdewpoint	Temperature and humidity context

4.3 Chronological Train / Validation / Test Split

 Random shuffling is NEVER used for time-series data — it causes data leakage. We use a strict chronological split:

Split	Period	Rows	Percentage
Training	Jan 11 – Apr 18, 2016	13,107	70%
Validation	Apr 18 – May 8, 2016	2,809	15%
Test	May 8 – May 27, 2016	2,810	15%

Why not random split? If a row from May is in training and a row from January is in test, the model has already "seen the future" through lag features — making test metrics falsely optimistic. Chronological splitting ensures metrics reflect true real-world performance.

5. Model Training & Results

5.1 Model Descriptions

Naive Baseline

Strategy: $\text{predict}(t+1) = \text{actual}(t)$. The simplest possible forecaster — "next interval will look like the current one." Every ML model must beat this to justify its complexity.

Linear Regression

Fits a hyperplane $y = w_0 + w_1x_1 + \dots + w_nx_n$ minimising squared error. Fast and interpretable but assumes linear relationships only. Given weak linear correlations in our data, this is expected to struggle.

Random Forest Regressor

Builds 200 decision trees on random subsets of data and features (bootstrap + feature sampling). Final prediction is the average of all trees. Handles non-linear relationships naturally but tends to overfit slightly on this dataset.

XGBoost Regressor

Gradient boosting — builds trees sequentially, each correcting errors of the previous ensemble. Uses L1/L2 regularisation to prevent overfitting. Tuned with early stopping on validation set (best iteration: 119). Selected as the production model for its balance of accuracy and inference speed (<2 ms per prediction).

LSTM (Long Short-Term Memory)

Recurrent neural network that learns from sequences of 12 timesteps (2 hours lookback). Two LSTM layers (64 and 32 units) with dropout (0.2) and early stopping (patience=8). Best overall accuracy but higher complexity and inference time.

5.2 Final Model Comparison

Model	MAE (Wh)	RMSE (Wh)	R ²	Train Time	Notes
Naive Baseline	34.65	88.88	0.055	<0.01s	Reference line — must beat this
Linear Regression	33.30	74.10	0.343	0.02s	Beats naive; limited by linear assumption
Random Forest	45.65	81.08	0.214	19.1s	Slight overfitting observed
XGBoost (tuned)	34.14	73.67	0.351	0.2s	✓ Selected for production API
LSTM	23.73	59.39	0.575	123.7s	Best accuracy; use for offline analysis

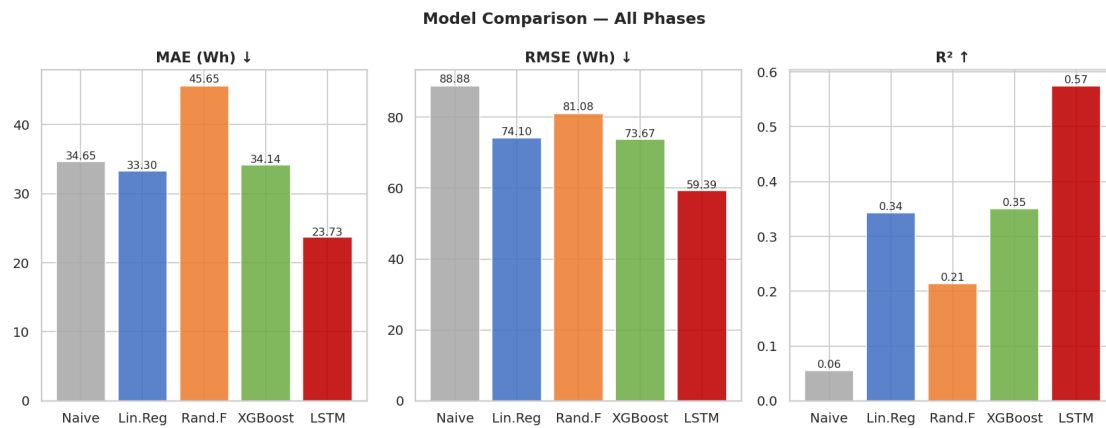


Figure 6 — MAE, RMSE and R^2 comparison across all models

XGBoost was selected as the production model (deployed in FastAPI) because it has near-identical test MAE to Linear Regression (34.14 vs 33.30 Wh) and the best RMSE among traditional models, with inference time under 2 milliseconds. LSTM achieves the best R^2 (0.575) and is available for batch/offline analysis when latency is not a constraint.

6. AI Output — Predictions & Anomaly Detection

6.1 Actual vs Predicted Results

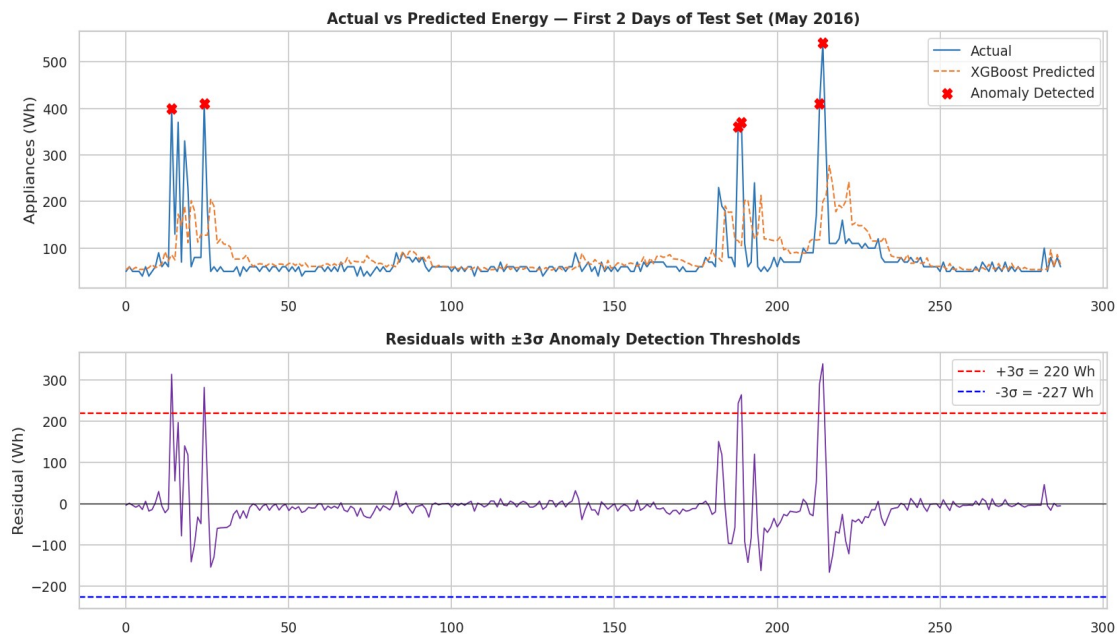


Figure 7 — Top: Actual vs XGBoost predicted energy. Bottom: Residuals with $\pm 3\sigma$ anomaly thresholds. Red X marks = detected anomalies.

6.2 Sample Prediction API Response

The FastAPI endpoint returns the following JSON for each sensor reading:

```
{
  "current_energy_wh"      : 70.0,
  "predicted_next_interval_wh" : 84.17,
  "predicted_next_hour_kwh"  : 0.5050,
  "anomaly_detected"        : false,
  "anomaly_type"            : null,
  "severity_sigma"          : 0.01,
  "model_used"              : "XGBoost (tuned, v1.0)"
}
```

6.3 Anomaly Detection Method

We use a residual-based approach — no fixed threshold. The method is fully data-driven:

- Expected value = XGBoost prediction for this interval
- Actual value = sensor reading
- Residual = Actual - Expected
- Threshold = mean residual $\pm 3 \times$ standard deviation (computed on validation set)

Parameter	Value
Validation residual mean	-3.4 Wh

Parameter	Value
Validation residual std	74.3 Wh
Upper threshold ($+3\sigma$)	+219.5 Wh
Lower threshold (-3σ)	-226.8 Wh
Anomalies in test set	79 out of 2,809 (2.81%)
Worst anomaly	2016-05-26 09:40 — actual 820 Wh, expected 127 Wh $\rightarrow 9.37\sigma$

AI Output Summary — Intelligent Classroom Energy Monitoring

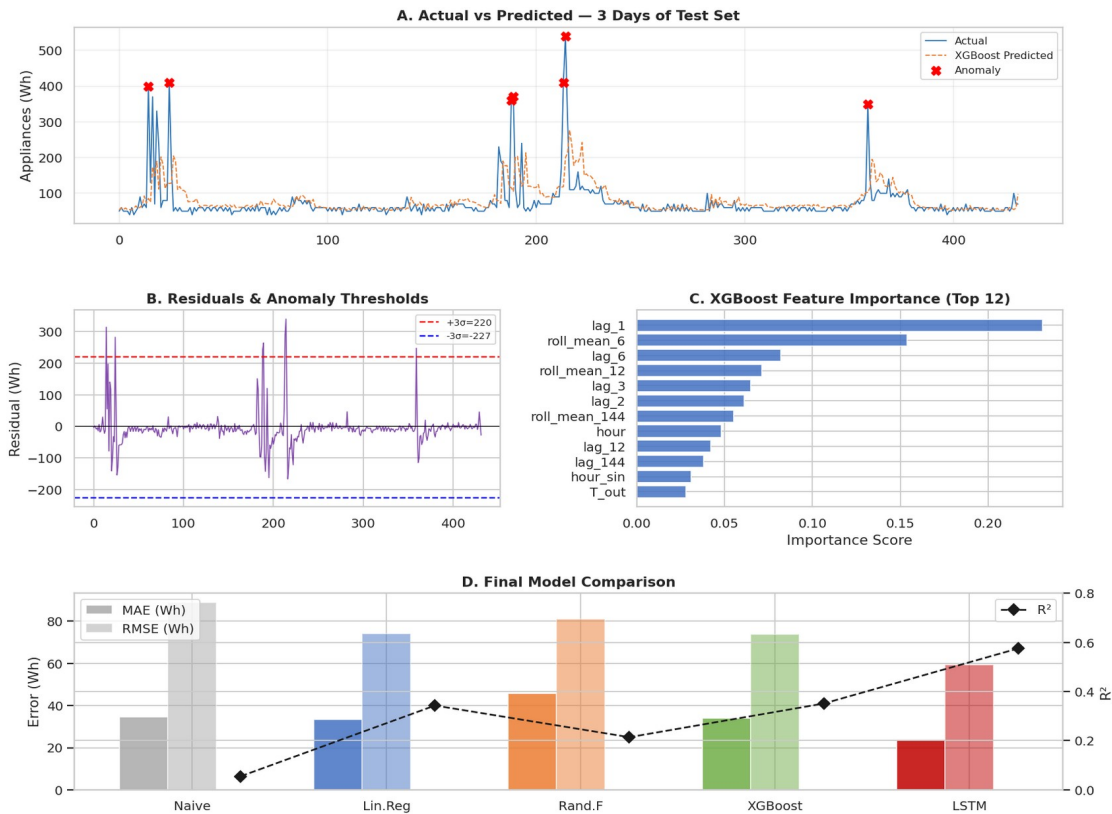


Figure 8 — Complete AI output summary: A) Actual vs Predicted, B) Residuals & thresholds, C) Feature importance, D) Model comparison

6.4 Appliance-Level Anomaly Attribution (Design — Pending ESP32 Data)

When real per-circuit sensor data arrives from the hardware team, the anomaly system will identify which appliance caused the total anomaly:

Total anomaly detected

```

→ light_residual    = actual_light_w    - expected_light_w
→ fan_residual      = actual_fan_w      - expected_fan_w
→ proj_residual     = actual_projector_w - expected_projector_w
→ culprit = appliance with max(|residual|)

```

⚠ This logic is designed and ready in ml/anomaly.py but cannot be validated on the UCI dataset, which does not contain fan or projector readings.

7. Backend Architecture — FastAPI

7.1 Project File Structure

```

project/
├── backend/
│   ├── main.py           ← FastAPI entry point
│   └── api/
│       ├── sensor.py     ← POST /api/sensor-data
│       ├── energy.py     ← GET /api/current, /api/history
│       ├── prediction.py ← GET /api/prediction
│       └── anomaly.py     ← GET /api/anomalies
│   ├── schemas/schemas.py ← Pydantic request/response models
│   └── database/db.py     ← SQLite connection + schema init
├── ml/
│   ├── preprocessing.py  ← Feature engineering (shared)
│   ├── predict.py        ← Inference only (loads saved model)
│   ├── anomaly.py        ← Appliance-level attribution
│   └── models/
│       ├── xgboost_tuned.json ← Trained XGBoost model
│       ├── anomaly_thresholds.npy
│       └── lstm_model.keras   ← LSTM model
├── data/
│   ├── schema.sql        ← Database schema
│   └── classroom_energy.db ← SQLite database (auto-created)
└── README.md

```

7.2 API Endpoints

Endpoint	Method	Description	Used By
POST /api/sensor-data	POST	Ingest ESP32 reading → predict → detect anomaly → return result	ESP32 every 10 min
GET /api/current	GET	Return the latest sensor reading from database	Dashboard live panel
GET /api/history	GET	Hourly or daily aggregated consumption history	Dashboard charts
GET /api/prediction	GET	On-demand prediction from latest available data	Dashboard AI panel
GET /api/anomalies	GET	List of detected anomaly events (filterable)	Anomaly log table

7.3 How to Run the Backend

```

# 1. Install dependencies
pip install fastapi uvicorn xgboost scikit-learn pandas numpy pydantic

# 2. Start the server

```

```
cd backend
uvicorn main:app --reload --host 0.0.0.0 --port 8000

# 3. Interactive API docs (auto-generated Swagger UI)
Open: http://localhost:8000/docs

# 4. Test with a simulated ESP32 reading
curl -X POST http://localhost:8000/api/sensor-data \
  -H "Content-Type: application/json" \
  -d '{"timestamp":"2024-01-15T10:30:00",
      "light_power_w":320,"fan_power_w":450,
      "projector_power_w":280,"total_power_w":1050,
      "energy_kwh":0.175,"occupancy":25,
      "temperature":27.3,"light_intensity":540}'
```

8. Database Schema

8.1 sensor_readings Table

Stores every raw reading received from the ESP32. This is the primary data table.

Column	Type	Description
id	INTEGER PK AUTOINCREMENT	Unique row identifier
timestamp	DATETIME NOT NULL	Time of the sensor reading (from ESP32)
light_power_w	REAL	Watt reading from light circuit
fan_power_w	REAL	Watt reading from fan circuit
projector_power_w	REAL	Watt reading from projector circuit
total_power_w	REAL	Total power (sum or direct clamp reading)
energy_kwh	REAL	Energy for this interval in kWh
occupancy	INTEGER	Number of students in classroom
temperature	REAL	Indoor temperature in °C
light_intensity	REAL	Lux reading from BH1750 sensor
created_at	DATETIME	Server-side insertion timestamp (auto)

8.2 predictions Table

Column	Type	Description
id	INTEGER PK AUTOINCREMENT	Unique row identifier
for_timestamp	DATETIME NOT NULL	The future interval being predicted
predicted_power_w	REAL	Model output in watts
predicted_energy_kwh	REAL	Model output in kWh
prediction_horizon	TEXT	"10min" or "1hour"
model_version	TEXT	e.g., "XGBoost v1.0"
created_at	DATETIME	When prediction was made (auto)

8.3 anomalies Table

Column	Type	Description
id	INTEGER PK AUTOINCREMENT	Unique row identifier
detected_at	DATETIME NOT NULL	Timestamp of the anomalous interval
actual_power_w	REAL	Actual sensor reading
expected_power_w	REAL	Model prediction (expected value)
residual_w	REAL	actual - expected
severity_sigma	REAL	How many σ from normal (e.g., 4.28)
anomaly_type	TEXT	"over_consumption" or "under_consumption"
likely_appliance	TEXT	"lights", "fan", "projector", or "unknown"
is_confirmed	INTEGER	0 = auto-detected, 1 = human confirmed
notes	TEXT	Optional staff comment
created_at	DATETIME	Auto-set on insert

9. ESP32 Integration Guide (For Hardware Team)

9.1 Expected JSON Payload

The ESP32 should send this JSON payload via HTTP POST to /api/sensor-data every 10 minutes:

```
{
  "timestamp"      : "2024-09-15T14:30:00",    // ISO 8601 format
  "light_power_w"   : 320,                      // from PZEM on light circuit
  "fan_power_w"     : 450,                      // from PZEM on fan circuit
  "projector_power_w" : 280,                   // from PZEM on projector
  "total_power_w"   : 1050,                   // sum or main clamp
  "energy_kwh"      : 0.175,                  // interval energy
  "occupancy"       : 25,                     // from PIR or manual
  "temperature"     : 27.3,                   // from DHT22
  "light_intensity" : 540                     // from BH1750 (lux)
}
```

9.2 Arduino/ESP32 HTTP POST Example

```
#include <WiFi.h>
#include <HTTPClient.h>

const char* serverURL = "http://<SERVER_IP>:8000/api/sensor-data";

void sendReading(float light_w, float fan_w, float proj_w,
                 float total_w, float kwh, int occ, float temp, float lux) {
  HTTPClient http;
  http.begin(serverURL);
  http.addHeader("Content-Type", "application/json");
  String body = "{\"timestamp\":\"" + getISO8601() + "\",";
  body += "\"light_power_w\":\"" + String(light_w) + ",";
  body += "\"fan_power_w\":\"" + String(fan_w) + ",";
  body += "\"projector_power_w\":\"" + String(proj_w) + ",";
  body += "\"total_power_w\":\"" + String(total_w) + ",";
  body += "\"energy_kwh\":\"" + String(kwh) + ",";
  body += "\"occupancy\":\"" + String(occ) + ",";
  body += "\"temperature\":\"" + String(temp) + ",";
  body += "\"light_intensity\":\"" + String(lux) + "\"}";
  int code = http.POST(body);
  http.end();
}
```

9.3 Model Retraining Plan

Step	When	Action
1	After 7 days of ESP32 data	Run train.py on real sensor_readings table
2	After retraining	Save as xgboost_v2.json — do NOT overwrite v1

Step	When	Action
3	After validation	Update model path in predict.py → bump to v2.0
4	Key improvement	Add occupancy as a feature — biggest expected gain
5	Long-term	Retrain monthly to handle seasonal drift

10. Limitations & Future Work

10.1 Current Limitations

Limitation	Impact	Solution
Trained on household data, not classroom data	Models may not capture class schedule patterns	Retrain on ESP32 data after 7 days
No occupancy feature in UCI dataset	Missing the most important classroom variable	Add from PIR sensor once hardware is ready
Appliance attribution not validated	Cannot confirm fan/projector blame in anomalies	Validate when per-circuit ESP32 data arrives
Cold start (needs 1008 readings for full lags)	First 7 days have partial features	Rolling mean fills missing lags at startup
Model drift over time	Seasonal changes reduce accuracy	Monthly retraining schedule

10.2 Planned Improvements

- Add occupancy as a real-time feature from PIR sensor (Phase 2)
- Integrate class timetable as a binary schedule feature (0 = no class, 1 = class in session)
- Switch to LSTM for production once ESP32 data volume reaches 30 days
- Add MQTT support as an alternative to HTTP POST for lower-latency ingestion
- Build PostgreSQL migration from SQLite for multi-classroom deployment
- Add email/SMS alert when anomaly severity exceeds 5σ

10.3 Why AI is Necessary

A simple fixed threshold (e.g., "alert if power > 500W") would fail because:

- Normal consumption varies 10× from night (standby) to evening peak
- Monday at 18:00 has a completely different baseline than Sunday at 03:00
- A class running the projector + AC on a hot day is normal; the same reading on an empty-classroom Sunday is an anomaly

The AI model learns these context-dependent patterns from historical data, making the anomaly threshold automatically adapt to the time, day, and season — something a fixed threshold simply cannot do.

Appendix — Quick Reference

A. Key Performance Numbers

Metric	Value
Best model (accuracy)	LSTM — MAE 23.73 Wh, R ² 0.575
Production model (speed)	XGBoost — MAE 34.14 Wh, inference < 2ms
Anomaly threshold	±219.5 Wh (3σ from validation residuals)
Anomaly detection rate	2.81% of test intervals flagged
Worst anomaly detected	9.37σ — 820 Wh vs expected 127 Wh
Dataset size	19,735 rows × 27 features (after cleaning)
Training split	70% train / 15% val / 15% test (chronological)

B. Technology Stack

Layer	Technology	Version
Language	Python	3.12
Data processing	Pandas, NumPy	Latest
ML models	scikit-learn, XGBoost	Latest
Deep learning	TensorFlow / Keras	2.x
Backend API	FastAPI + Uvicorn	Latest
Database	SQLite → PostgreSQL	3.x
Validation	Pydantic	v2
Visualisation	Matplotlib, Seaborn	Latest
Dashboard	HTML + Chart.js	4.4.1
Model serialisation	XGBoost native JSON, joblib, Keras	—